

Prog. Nodos Sensores IoT

Curso 2023-2024

1 de Noviembre- 30 de Noviembre
de 2023

Práctica 4: Programación de sensores mediante hilos y comunicación MQTT

Guillermo Botella

Objetivos

- Uso de dispositivos conectados por un bus serie (I2C, SPI).
- Uso de sensor de infrarrojos.
- Extraer la información necesaria a partir de datasheets de componentes.
- Implementación estructurada mediante máquina de estados con una implementación de hilos que realicen funcionalidades distintas.

Requisitos



- Para poder realizar con éxito la práctica, es necesario revisar conceptos vistos en clase:
 - Interfaz de uso de buses serie proporcionada por WiringPi (I2C esencialmente)
 - Lectura de *datasheets* de los componentes utilizados, comprendiendo su funcionamiento.
 - Manipulación de variables a nivel de bit en C.

Desarrollo de la práctica

- Utilizaremos una unidad de movimiento basada en sensor de temperatura disponible en Pololu LSM6DS33.
- Además se utilizará el sensor de infrarrojos de SHARP GP2Y0A41SK0 que nos permite medir distancias en un rango de 4cm a 30cm.
- Los datasheets de los componentes utilizados están disponibles en el Campus Virtual.

Hilos (threads)

- Un proceso GNU/Linux pueden definirse varios hilos de ejecución con ayuda de la biblioteca *libpthread*, que permite usar un conjunto de funciones que siguen el estándar POSIX.
- Todo proceso contiene un hilo inicial (main) cuando comienza a ejecutarse. A continuación pueden crearse nuevos hilos con:

```
int pthread_create(pthread_t *tid,  
    const pthread_attr_t *attr,  
    void *(*func)(void *), void *arg);
```

- Compilación:

Terminal #1

```
user@lab:~ $ gcc -o exec threads_example.c -lpthread
```

Hilos (threads)

```
int pthread_create(pthread_t *tid, const
                  pthread_attr_t *attr,
                  void *(*func)(void *), void *arg);
```

- “pthread_t *tid”: Un puntero a una variable donde se almacenará el identificador del nuevo hilo.
- “const pthread_attr_t *attr”: Un puntero a una estructura con atributos del hilo, que puede especificar preferencias como el tamaño de la pila, la política de planificación, etc. Si es `NULL`, se usan los atributos por defecto.
- “void *(*func)(void *)”: Un puntero a la función que el hilo ejecutará.
- “void *arg”: Un puntero a los argumentos que se pasan a la función que el hilo ejecutará.

Hilos



```
int pthread_create(pthread_t *tid,  
    const pthread_attr_t *attr,  
    void *(*func)(void *), void *arg);
```

donde:

- `tid` es el identificador de hilo que la llamada devuelve.
- `attr` son los atributos de creación del hilo (tamaño de pila, política de planificación, estado de separación,...).
- `func` es la función definida por el usuario que será ejecutada por el nuevo hilo. Debe admitir un argumento `void *` y debe devolver un valor `void *`.
- `arg` es el argumento que se le pasa a `func`.

Hilos (hello world)

hello.c

```
#include <stdio.h>
#include <pthread.h>
// remember to set compilation option -pthread

void *myfunc(void *msg) {
    printf("msg: %s\n", msg);
    return NULL;
}

int main() {
    pthread_t tid1, tid2;
    pthread_create(&tid1, NULL, myfunc, "Jabberwocky");
    pthread_create(&tid2, NULL, myfunc, "Vorpel");

    // wait for both threads to finish :
    void* result;
    pthread_join(tid1, &result);
    pthread_join(tid2, &result);
}
```

Terminal #1

```
user@lab:~ $ gcc -o hello hello.c -lpthread
user@lab:~ $ ./hello
msg: Jabberwocky
msg: Vorpel
```

Tarea

- El programa **cthilos.c** recibe *arg1 ...argMax* argumentos y ejecuta **n** hilos, cada uno de los cuales recibe como argumento un *argi* y escribe en pantalla un número aleatorio de veces el argumento recibido.
- Comprobar la compilación ejecutando **make** y comprender el funcionamiento del código.

Infrarrojos

- Sensor de infrarrojos mide distancias con un rango de entre 4cm y 30cm.
- Importante: necesario usar un ADC para leer los valores.

Nota

- De la práctica 2, para usar el ADC es necesario:
 - Configurar las conexiones para trabajar con el bus SPI
 - Convertir el valor leído (entero de 10 bits) al voltaje medido (sabiendo la tensión de referencia 3.3V).
- Traducir el voltaje medido a la distancia de acuerdo a las gráficas del *datasheet*.

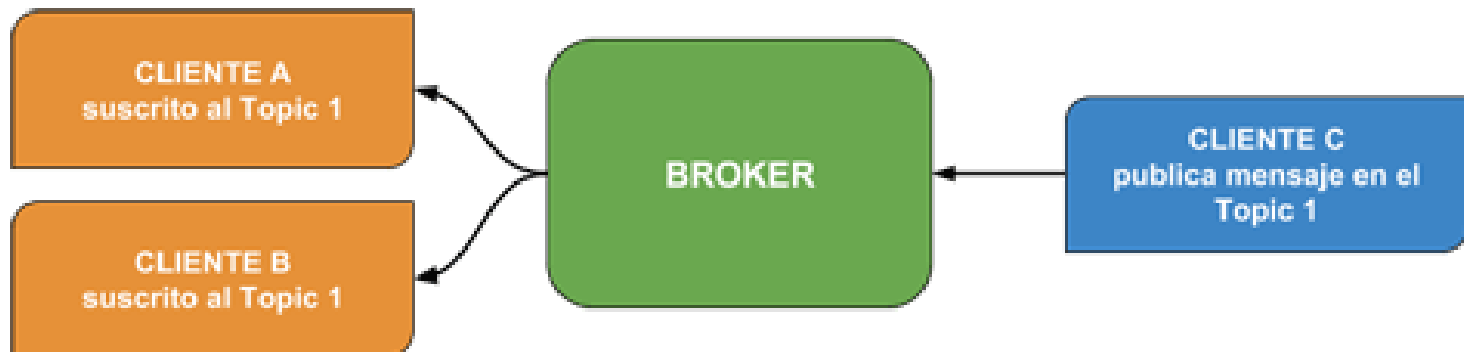
Tarea

- Muestrear el sensor de infrarrojos con una frecuencia de 1Hz.

MQTT



- MQTT es un protocolo M2M (Message Queue Telemetry Transport).
- Basado en un protocolo de mensajería publicación/suscripción, al contrario que HTTP que es petición/respuesta.
- Características:
 - Minimizar el ancho de banda.
 - Comunicación bidireccional entre dispositivos.
 - Minimizar los requerimientos de consumo.
 - Garantizar la fiabilidad y cierto grado de seguridad.



MQTT



- Un cliente se suscribe a un topic del **broker** que es como el "tema", a quién va dirigido el mensaje.
- El topic es el tema del mensaje.
 - Los topics se organizan de acuerdo a un jerarquía de mensaje separada por niveles referido al carácter /
- Existen comodines como el símbolo + y #.
 - El símbolo + se sustituye por cualquier nivel. Ej: todos los topic temperatura /casa/+/temperatura
 - El símbolo # permite acceder a varios niveles aguas abajo. Ej: todos los mensajes que se envían a casa: /casa/#

Mosquitto en Rpi

- Existe una implementación de MQTT para Raspberry Pi: **Eclipse Mosquitto** disponible en <https://mosquitto.org/>

Instalación

```
apt-get install mosquitto-clients
```

Suscripción

- Para un tópico se utiliza en comando *mosquitto_sub*:

```
mosquitto_sub -h BROKER -t TOPIC
```

Publicación

- Para publicación de un *MENSAJE* en el tópico:

```
mosquitto_pub -h BROKER -t TOPIC -m MENSAJE
```

Tarea MQTT

- Desarrollar el ejemplo realizado en clase para publicar y subscribir tópicos en el broker `test.mosquitto.org` en lugar de `localhost`

Desarrollo en C

- En el repositorio de código de la implementación de Mosquitto hay ejemplos de uso
- El fichero **basic-1.c** disponible en el repositorio git <https://github.com/eclipse/mosquitto/tree/master/examples/publish/basic-1.c>

Tarea

- Comprender el ejemplo de publicación **basic-1.c** y adaptarlo para publicar cualquier mensaje

Desarrollo en C

- El ejemplo de publicación **basic-1.c** publica un tópico en la rama “example/temperature” del servidor de mosquito **test.mosquitto.org** a través del puerto 1883:
 - 1 Inicialización de Mosquitto con `mosquitto_lib_init`
 - 2 Creación del cliente con `mosquitto_new`, y configuración de *callbacks*
 - 3 Conexión al broker con el comando `mosquitto_connect`
 - 4 Publicación del topic en la función `publish_sensor_data` que a su vez invoca a la función de publicación del tópico `mosquitto_publish`, donde se envía el tópico en el campo *topic* y el texto mediante la cadena de caracteres *payload*
 - 5 Finalización del cliente con el comando `mosquitto_lib_cleanup`

Instalación

```
sudo apt-get install libmosquitto-dev
```

Desarrollo de la práctica

- Tres programas a desarrollar (museo con obras de arte).
- 1 Nodo Sensorización** de temperatura (Pololu) y distancia de visitantes (SHARP) y publicación en rama del servidor.
 - Hilo de sensorización de distancia a 10Hz.
 - Hilo de temperatura a 0.1 Hz.
- 2 Nodo actuador** de alarma que se suscribe a los tópicos distancia: encendido de alarma (led) si distancia menor a 10cm.
- 3 Nodo climatizador** que calcule la media de los medidas de temperatura (tópico temperatura), y activa climatización (led).

Prog. Nodos Sensores IoT

Curso 2023-2024

1 de Noviembre- 30 de Noviembre
de 2023

Práctica 4: Programación de sensores mediante hilos y comunicación MQTT

Guillermo Botella